Day 5 - JavaScript - R Veera Brahmanandam - Hands on Tasks

Problem 1: Greeting Generator (Level-1)

Scenario

A simple webpage should greet users based on the name they enter. This helps beginners understand how JavaScript functions accept parameters and how variables behave inside functions.

Requirements

- Create an input box to enter a user's name.
- Create a button labelled "Generate Greeting".
- When the button is clicked: A JavaScript function should accept the name as a parameter.
- Display a greeting message like: "Hello, Rahul! Welcome to our website."
- Display the greeting inside a <p> element.

Complete Code:

```
<!-- Day 5 - JavaScript - R Veera Brahmanandam - Hands On -->
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Greeting Generator</title>
</head>
<body style="
   background: linear-gradient(135deg, #667eea, #764ba2);
   font-family: Arial, Helvetica, sans-serif;
   height: 100vh;
   margin: 0;
   display: flex;
   justify-content: center;
   align-items: center;
">
   <!-- Main container -->
```

```
<div style="
    background-color: #ffffff;
    padding: 30px 40px;
    border-radius: 12px;
    box-shadow: 0px 10px 25px rgba(0,0,0,0.2);
    text-align: center;
    width: 350px;
">
    <!-- Heading -->
    <h1 style="
        color: #4a148c;
        margin-bottom: 20px;
    ">
        Greeting Generator
    </h1>
    <!-- Input box -->
    <input
        type="text"
        id="username"
        placeholder="Enter user's name"
        style="
            width: 320px;
            padding: 12px;
            font-size: 16px;
            border-radius: 8px;
            border: 2px solid #7b1fa2;
            outline: none;
            margin-bottom: 15px;
        "
    >
    <!-- Button -->
    <button
        onclick="generateGreeting()"
```

```
        style="
            background: linear-gradient(135deg, #ff6a00, #ee0979);
            color: white;
            padding: 12px 20px;
            font-size: 16px;
            border: none;
            border-radius: 8px;
            cursor: pointer;
            width: 100%;
            font-weight: bold;
        "
    >
        Generate Greeting
    </button>
    <!-- Output paragraph -->
    <p
        id="greeting"
        style="
            margin-top: 20px;
            font-size: 18px;
            color: #1b5e20;
            font-weight: bold;
        "
    ></p>
</div>
<script>
    // Use function keyword
    function generateGreeting() {
        // Javascript function should accept the name as a parameter
        var name = document.getElementById("username").value;
        // Variable for the greeting message must be declared inside the function
        var message = "Hello, " +name+ "! Welcome to our websites.";
        // Use document.getElementById() for DOM manipulation
```

```
        document.getElementById("greeting").innerHTML = message;
    }
  </script>
</body>
</html>
```

Code Screenshot:

Task1 - Greeting Generator > Greeting_Generator.html > html > body > div

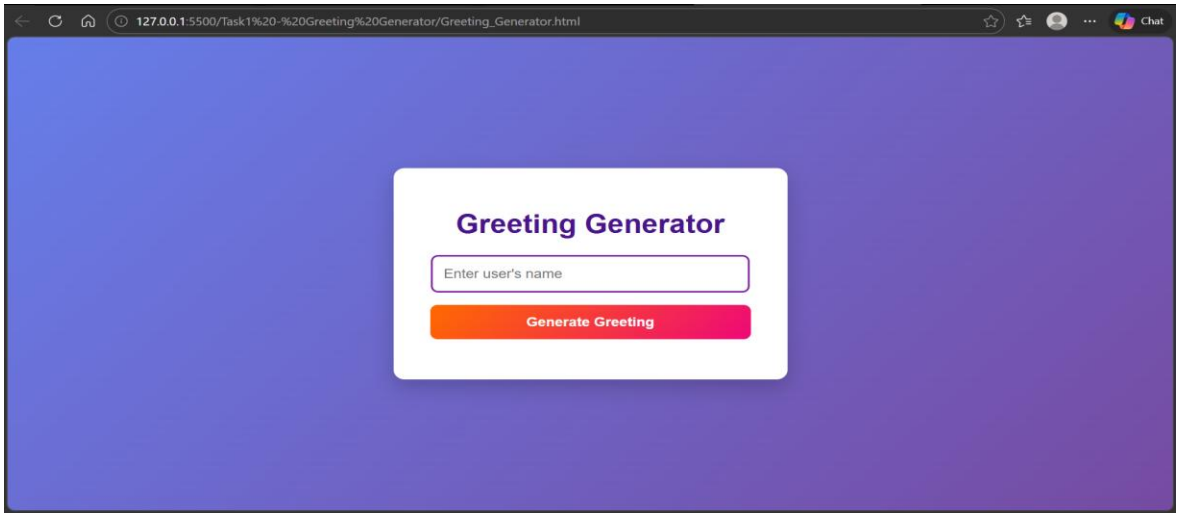
```html
<!-- Day 5 - JavaScript - R Veera Brahmanandam - Hands On -->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Greeting Generator</title>
</head>

<body style="
    background: linear-gradient(135deg, #667eea, #764ba2);
    font-family: Arial, Helvetica, sans-serif;
    height: 100vh;
    margin: 0;
    display: flex;
    justify-content: center;
    align-items: center;
">

    <!-- Main container -->
    <div style="
        background-color: #ffffff;
        padding: 30px 40px;
        border-radius: 12px;
        box-shadow: 0px 10px 25px rgba(0,0,0,0.2);
        text-align: center;
        width: 350px;
    ">

        <!-- Heading -->
        <h1 style="
            color: #4a148c;
            margin-bottom: 20px;
        ">
            Greeting Generator
        </h1>

        <!-- Input box -->
        <input
            type="text"
            id="username"
            placeholder="Enter user's name"
            style="
```

```html
            style="
                width: 320px;
                padding: 12px;
                font-size: 16px;
                border-radius: 8px;
                border: 2px solid #7b1fa2;
                outline: none;
                margin-bottom: 15px;
            "
        >

        <!-- Button -->
        <button
            onclick="generateGreeting()"
            style="
                background: linear-gradient(135deg, #ff6a00, #ee0979);
                color: white;
```

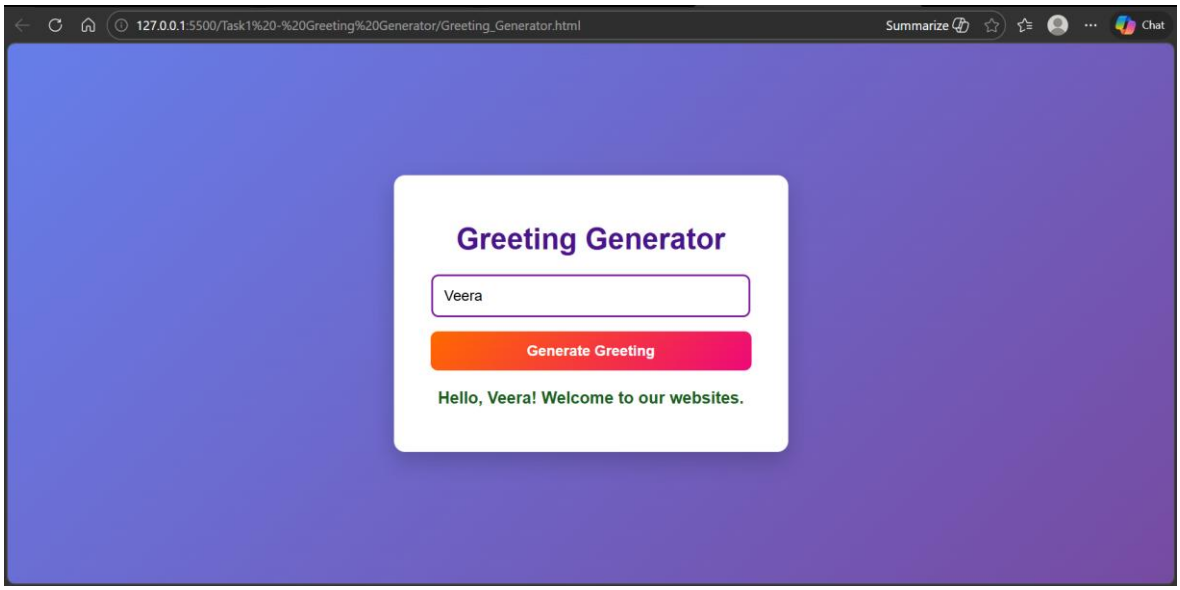
```html
                padding: 12px 20px;
                font-size: 16px;
                border: none;
                border-radius: 8px;
                cursor: pointer;
                width: 100%;
                font-weight: bold;
            "
        >
            Generate Greeting
        </button>

        <!-- Output paragraph -->
        <p
            id="greeting"
            style="
                margin-top: 20px;
                font-size: 18px;
                color: #1b5e20;
                font-weight: bold;
            "
        ></p>

    </div>

    <script>

        // Use function keyword

        function generateGreeting() {

            // Javascript function should accept the name as a parameter
            var name = document.getElementById("username").value;

            // Variable for the greeting message must be declared inside the function
            var message = "Hello, " +name+ "! Welcome to our websites.";

            // Use document.getElementById() for DOM manipulation
            document.getElementById("greeting").innerHTML = message;
        }
    </script>
</body>
</html>
```

Output Screenshot:

“The screenshot shows a simple Greeting Generator webpage.”



It contains a title at the top, a text input field where the name “Veera” is entered, and a button labelled “Generate Greeting.” After clicking the button, a personalized message appears below. The displayed message says **“Hello, Veera! Welcome to our websites.”**



<u>Problem 2:</u> Simple Object-Based User Info Display (Level-1)

Scenario

A webpage needs to display basic user information stored inside a JavaScript object.

Requirements

Create a JavaScript object user with properties:

- name
- age
- city

Create a function displayUserInfo(userObj):

- Accepts the object as a parameter.
- Displays user details in separate <p> elements.

A button click should trigger the function.

<u>Complete Code:</u>

```
<!-- Day 5 - JavaScript - R Veera Brahmanandam - Hands On Tasks -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>User Info Display</title>
<style>
    body {
        margin: 0;
        height: 100vh;
        display: flex;
        justify-content: center;
        align-items: center;
        font-family: Arial, sans-serif;
        background: linear-gradient(to right, #667eea, #764ba2);
    }
    .container {
        background-color: #ffffff;
        padding: 30px;
        width: 350px;
        border-radius: 12px;
        box-shadow: 0 10px 25px rgba(0,0,0,0.25);
        text-align: center;
    }
    h2 {
        margin-bottom: 20px;
        color: #333;
    }
    button {
        width: 100%;
        padding: 12px;
        background-color: #667eea;
        color: white;
        border: none;
        border-radius: 6px;
        font-size: 16px;
        cursor: pointer;
```

```
            margin-bottom: 20px;
        }
        button:hover {
            background-color: #5a67d8;
        }
        .info {
            display: none; /* hidden initially */
            text-align: left;
        }
        .info p {
            background-color: #f1f4ff;
            padding: 10px;
            border-radius: 6px;
            margin: 10px 0;
            font-size: 16px;
            color: #333;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>User Information</h2>
        <button onclick="displayUserInfo(user)">Show User Info</button>
        <!-- Details appear BELOW the button -->
        <div class="info" id="userInfo">
            <p id="name"></p>
            <p id="age"></p>
            <p id="city"></p>
        </div>
    </div>
    <script>
        // JavaScript Object
        const user = {
```

```
        name: "Veera",
        age: 22,
        city: "Hyderabad"
    };
    function displayUserInfo(userObj) {
        document.getElementById("name").textContent = "Name: " + userObj.name;
        document.getElementById("age").textContent = "Age: " + userObj.age;
        document.getElementById("city").textContent = "City: " + userObj.city;
        // Show the info section
        document.getElementById("userInfo").style.display = "block";
    }
  </script>
</body>
</html>
```

Code Screenshot:

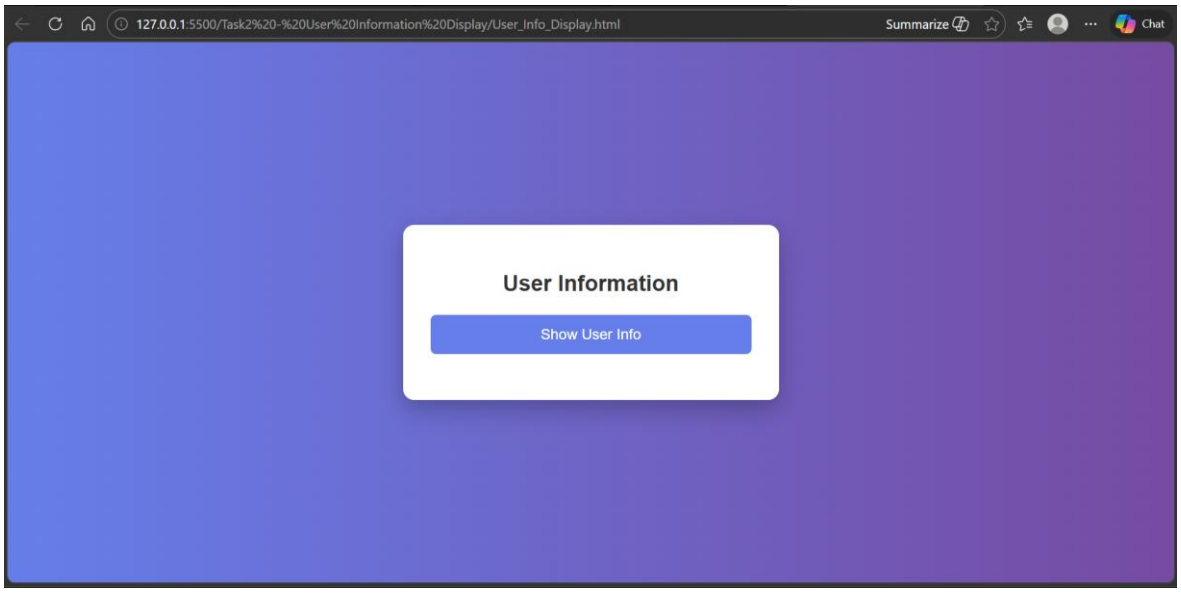
Task2 - User Information Display > User_Info_Display.html > html > head > style > .info p

```
<!-- Day 5 - JavaScript - R Veera Brahmanandam - Hands On Tasks -->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>User Info Display</title>

    <style>
        body {
            margin: 0;
            height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            font-family: Arial, sans-serif;
            background: linear-gradient(to right, #667eea, #764ba2);
        }

        .container {
            background-color: #ffffff;
            padding: 30px;
            width: 350px;
            border-radius: 12px;
            box-shadow: 0 10px 25px rgba(0,0,0,0.25);
            text-align: center;
        }

        h2 {
            margin-bottom: 20px;
            color: #333;
        }

```

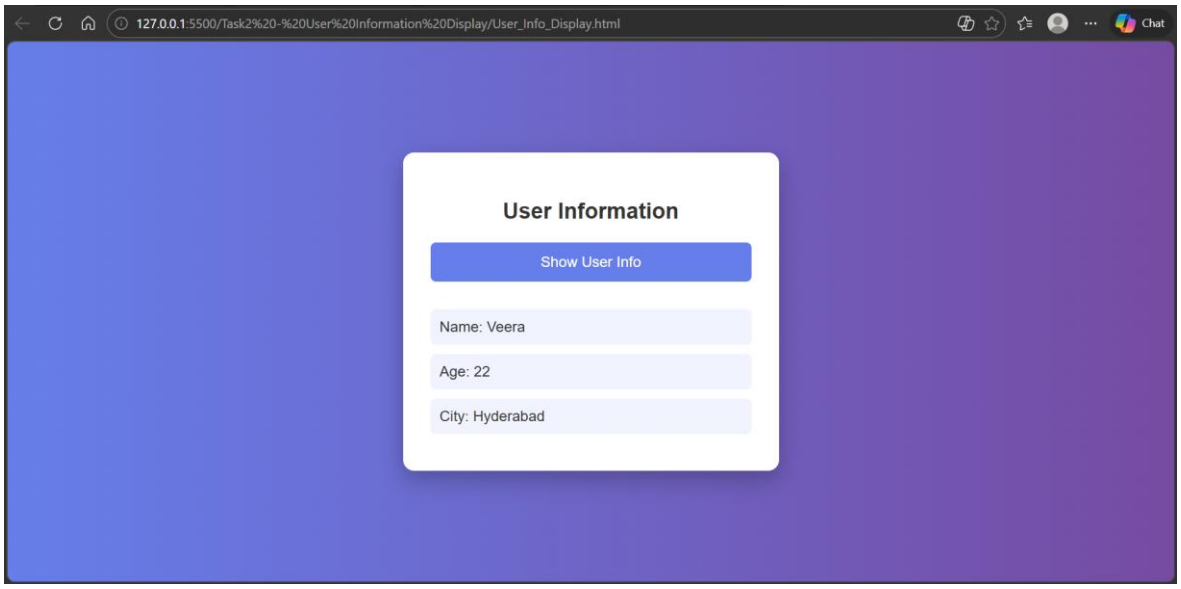
```html
        button {
            width: 100%;
            padding: 12px;
            background-color: #667eea;
            color: white;
            border: none;
            border-radius: 6px;
            font-size: 16px;
            cursor: pointer;
            margin-bottom: 20px;
        }

        button:hover {
            background-color: #5a67d8;
        }

        .info {
            display: none; /* hidden initially */
            text-align: left;
        }

        .info p {
            background-color: #f1f4ff;
            padding: 10px;
            border-radius: 6px;
            margin: 10px 0;
            font-size: 16px;
            color: #333;
        }
    </style>
</head>
<body>

    <div class="container">
        <h2>User Information</h2>

        <button onclick="displayUserInfo(user)">Show User Info</button>

        <!-- Details appear BELOW the button -->
        <div class="info" id="userInfo">
            <p id="name"></p>
            <p id="age"></p>
            <p id="city"></p>
        </div>
    </div>

    <script>
        // JavaScript Object
        const user = {
            name: "Veera",
            age: 22,
            city: "Hyderabad"
        };

        function displayUserInfo(userObj) {
            document.getElementById("name").textContent = "Name: " + userObj.name;
            document.getElementById("age").textContent = "Age: " + userObj.age;
            document.getElementById("city").textContent = "City: " + userObj.city;

            // Show the info section
            document.getElementById("userInfo").style.display = "block";
        }
    </script>

</body>
</html>
```

Output Screenshot:

The screenshot displays a simple webpage titled **"User Information"** It contains a button labelled **"Show User Info"**, which is designed to display user details when clicked.



When we click on the **"Show User Info"** button it displays the user information



Problem 3: Counter App with Scope Control (Level-2)

Scenario

Build a counter application where users can increment and reset a value. This problem focuses on variable scope and DOM manipulation.

Requirements: Display a counter value starting from 0.

Create two buttons:

- **Increment**
- **Reset**

Use:

- A global variable to store counter value.
- A function incrementCounter(step) that:
    - Accepts step value as a parameter.
    - Updates the counter.

Reset button should reset the counter to 0.

<u>Complete Code:</u>

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Counter App - Scope Control</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="counter-container">
        <h1>Counter App</h1>
        <!-- Display counter value starting from 0 -->
        <p id="counterValue">0</p>
        <!-- Buttons (No inline JavaScript used) -->
        <button id="incrementBtn">Increment</button>
        <button id="resetBtn">Reset</button>
    </div>
    <!-- External JavaScript file linked -->
    <script src="script.js"></script>
</body>
</html>
```

Script.js:

```
/*
 * GLOBAL SCOPE
 * Requirement
 * Counter value is maintained outside the function
 */
let counter = 0;
/*
 * Function: incrementCounter(step)
 * Requirement
 * - Accepts step value as a parameter
 * - Updates the counter value
 * - DOM update happens inside the function only
*/
const incrementCounter = (step) => {
    counter += step; // updating global variable
    document.getElementById("counterValue").textContent = counter;
};
/*
 * Function: resetCounter()
 * Requirement
 * - Resets counter value to 0
 * - DOM update inside function
*/
const resetCounter = () => {
    counter = 0;
    document.getElementById("counterValue").textContent = counter;
};
/*
 * Event Listeners
 * Requirement
 * - No inline JavaScript in HTML
 * - UI controlled using JavaScript functions
```

```
*/
document.getElementById("incrementBtn").addEventListener("click", () => {
    incrementCounter(1); // passing step value as parameter
});
document.getElementById("resetBtn").addEventListener("click", resetCounter);
```

style.css

```
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #667eea, #764ba2);
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
}
.counter-container {
    background-color: #ffffff;
    padding: 30px 40px;
    border-radius: 12px;
    text-align: center;
    box-shadow: 0 10px 25px rgba(0,0,0,0.2);
}
h1 {
    margin-bottom: 20px;
    color: #333;
}
#counterValue {
    font-size: 60px;
    font-weight: bold;
    color: #4a00e0;
    margin: 20px 0;
}
button {
```

```
    padding: 10px 20px;
    font-size: 16px;
    margin: 8px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
}
#incrementBtn {
    background-color: #4caf50;
    color: white;
}
#resetBtn {
    background-color: #f44336;
    color: white;
}
button:hover {
    opacity: 0.9;
}
```

Code Screenshot:

index.html

Task3 - Counter App with Scope Control > index.html > html > body > div.counter-container > button#resetBtn

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Counter App - Scope Control</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="counter-container">
        <h1>Counter App</h1>

        <!-- Display counter value starting from 0 -->
        <p id="counterValue">0</p>

        <!-- Buttons (No inline JavaScript used) -->
        <button id="incrementBtn">Increment</button>
        <button id="resetBtn">Reset</button>
    </div>

    <!-- External JavaScript file linked -->
    <script src="script.js"></script>
</body>
</html>
```

script.js

Task3 - Counter App with Scope Control > JS script.js > resetCounter

```javascript
/*
 * GLOBAL SCOPE
 * Requirement
 * Counter value is maintained outside the function
 */
let counter = 0;

/*
 * Function: incrementCounter(step)
 * Requirement
 * - Accepts step value as a parameter
 * - Updates the counter value
 * - DOM update happens inside the function only
 */
const incrementCounter = (step) => {
    counter += step; // updating global variable
    document.getElementById("counterValue").textContent = counter;
};

/*
 * Function: resetCounter()
 * Requirement
 * - Resets counter value to 0
 * - DOM update inside function
 */
const resetCounter = () => {
    counter = 0;
    document.getElementById("counterValue").textContent = counter;
};

/*
 * Event Listeners
 * Requirement
 * - No inline JavaScript in HTML
 * - UI controlled using JavaScript functions
 */
document.getElementById("incrementBtn").addEventListener("click", () => {
    incrementCounter(1); // passing step value as parameter
});

document.getElementById("resetBtn").addEventListener("click", resetCounter);
```

style.css

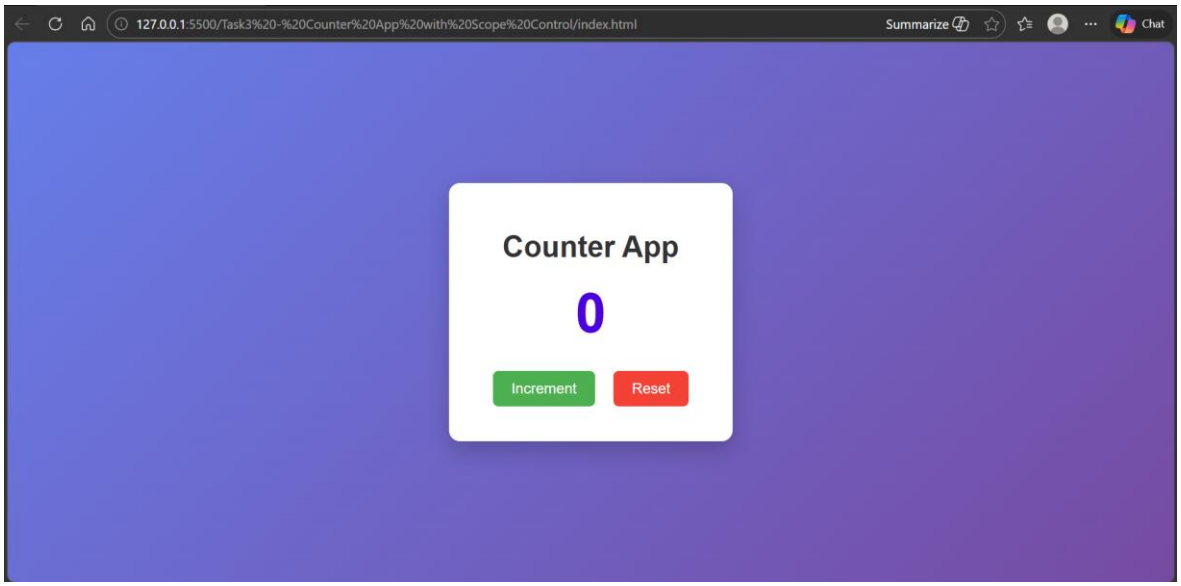
Task3 - Counter App with Scope Control > # style.css > #counterValue

```css
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #667eea, #764ba2);
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
}

.counter-container {
    background-color: #ffffff;
    padding: 30px 40px;
    border-radius: 12px;
    text-align: center;
    box-shadow: 0 10px 25px rgba(0,0,0,0.2);
}

h1 {
    margin-bottom: 20px;
    color: #333;
}
```

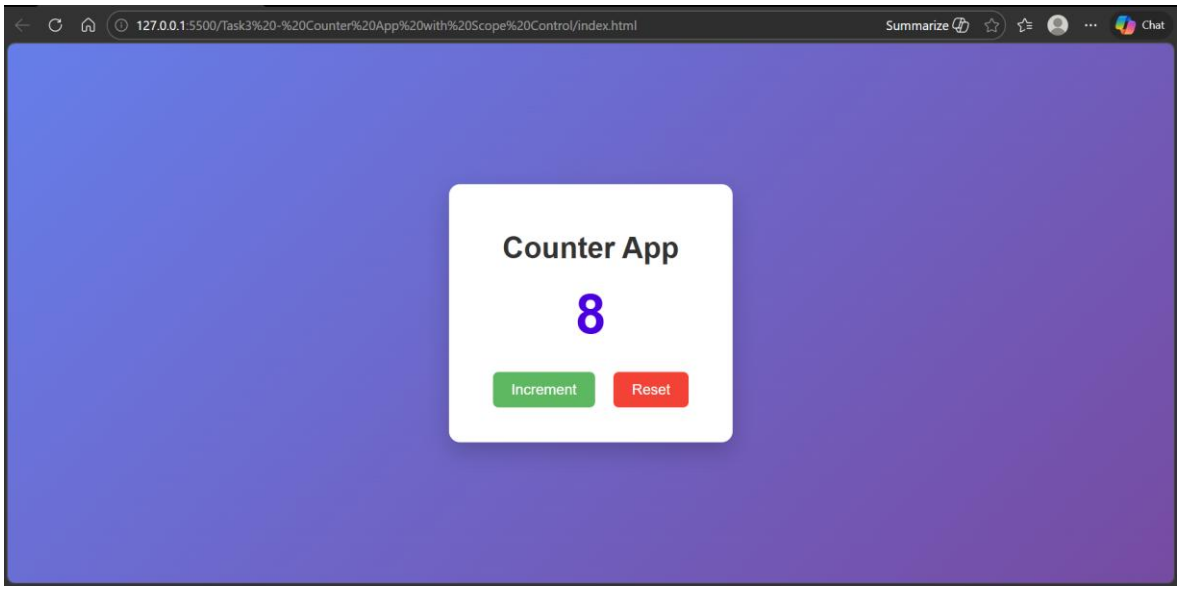
```css
#counterValue {
    font-size: 60px;
    font-weight: bold;
    color: #4a00e0;
    margin: 20px 0;
}

button {
    padding: 10px 20px;
    font-size: 16px;
    margin: 8px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
}

#incrementBtn {
    background-color: #4caf50;
    color: white;
}

#resetBtn {
    background-color: #f44336;
    color: white;
}

button:hover {
    opacity: 0.9;
}
```

Output Screenshot:

The screenshot shows a simple Counter App webpage. It displays the title **"Counter App"** and a number starting at 0. Below the number, there are two buttons labelled **"Increment"** and **"Reset."**



The Increment button increases the count, while the Reset button sets the counter back to zero.



Problem 4: Dynamic Student Profile Manager (Level-2)

Scenario

Create a mini student profile system where details are stored in an object and displayed dynamically using DOM manipulation.

Requirements

Create a student object with:

- name
- rollNo
- marks

Create a function updateStudentProfile(studentObj):

- Accepts the object as a parameter.
- Displays details in a styled .

Add another function updateMarks(newMarks):

- Updates marks and refreshes the display.

<u>Complete Code:</u>

Student_profile.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Dynamic Student Profile Manager</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Student Profile Manager</h1>
    <!-- Profile Display Area -->
    <!-- Display details dynamically using DOM -->
    <div id="profileBox"></div>
    <!-- Button to update marks -->
    <button onclick="updateMarks(95)">Update Marks</button>
    <script>
        /* GLOBAL STUDENT OBJECT: Student object must be in global scope */
        let student = {
            name: "Veera",
            rollNo: 101,
            marks: 88
        };
        /* FUNCTION: updateStudentProfile(studentObj)
           - Accepts object as parameter
           - Displays details dynamically
           - Uses DOM manipulation */
        function updateStudentProfile(studentObj) {
            document.getElementById("profileBox").innerHTML = `
                <h2>Student Details</h2>
                <p><strong>Name:</strong> ${studentObj.name}</p>
                <p><strong>Roll No:</strong> ${studentObj.rollNo}</p>
                <p><strong>Marks:</strong> ${studentObj.marks}</p>
            `;
        }
        /* FUNCTION: updateMarks(newMarks)
           - Updates marks using parameter
           - Modifies shared global object
           - Refreshes DOM without page reload */
```

```
        function updateMarks(newMarks) {
            student.marks = newMarks;  // object manipulation
            updateStudentProfile(student); // refresh UI
        }
        /* INITIAL DISPLAY
           - Display student details when page loads */
        updateStudentProfile(student);
    </script>
</body>
</html>
```

style.css:

```
/* Page styling */
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(to right, #74ebd5, #acb6e5);
    text-align: center;
    padding: 40px;
}
/* Heading */
h1 {
    color: #2c3e50;
    margin-bottom: 30px;
}
/* Student profile box */
#profileBox {
    background-color: #ffffff;
    width: 300px;
    margin: auto;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
}
```

```
/* Text inside profile */
#profileBox p {
    font-size: 16px;
    color: #34495e;
}
/* Button styling */
button {
    margin-top: 20px;
    padding: 10px 20px;
    background-color: #2ecc71;
    border: none;
    border-radius: 6px;
    color: white;
    font-size: 16px;
    cursor: pointer;
}
/* Button hover effect */
button:hover {
    background-color: #27ae60;
}
```

Code Screenshot:

Task4 - Dynamic Student Profile Manager > Student-Profile.html > html > body > script

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Student Profile Manager</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <h1>Student Profile Manager</h1>

    <!-- Profile Display Area -->
    <!-- Display details dynamically using DOM -->
    <div id="profileBox"></div>

    <!-- Button to update marks -->
    <button onclick="updateMarks(95)">Update Marks</button>

    <script>
        /* GLOBAL STUDENT OBJECT: Student object must be in global scope */
        let student = {
            name: "Veera",
            rollNo: 101,
            marks: 88
        };
```

```html
        /* FUNCTION: updateStudentProfile(studentObj)
           - Accepts object as parameter
           - Displays details dynamically
           - Uses DOM manipulation */

        function updateStudentProfile(studentObj) {
            document.getElementById("profileBox").innerHTML = `
                <h2>Student Details</h2>
                <p><strong>Name:</strong> ${studentObj.name}</p>
                <p><strong>Roll No:</strong> ${studentObj.rollNo}</p>
                <p><strong>Marks:</strong> ${studentObj.marks}</p>
            `;
        }

        /* FUNCTION: updateMarks(newMarks)
           - Updates marks using parameter
           - Modifies shared global object
           - Refreshes DOM without page reload */
        function updateMarks(newMarks) {
            student.marks = newMarks;  // object manipulation
            updateStudentProfile(student); // refresh UI
        }

        /* INITIAL DISPLAY
           - Display student details when page loads */
        updateStudentProfile(student);
    </script>

</body>
</html>
```

Style.css

Task4 - Dynamic Student Profile Manager > # style.css > button:hover

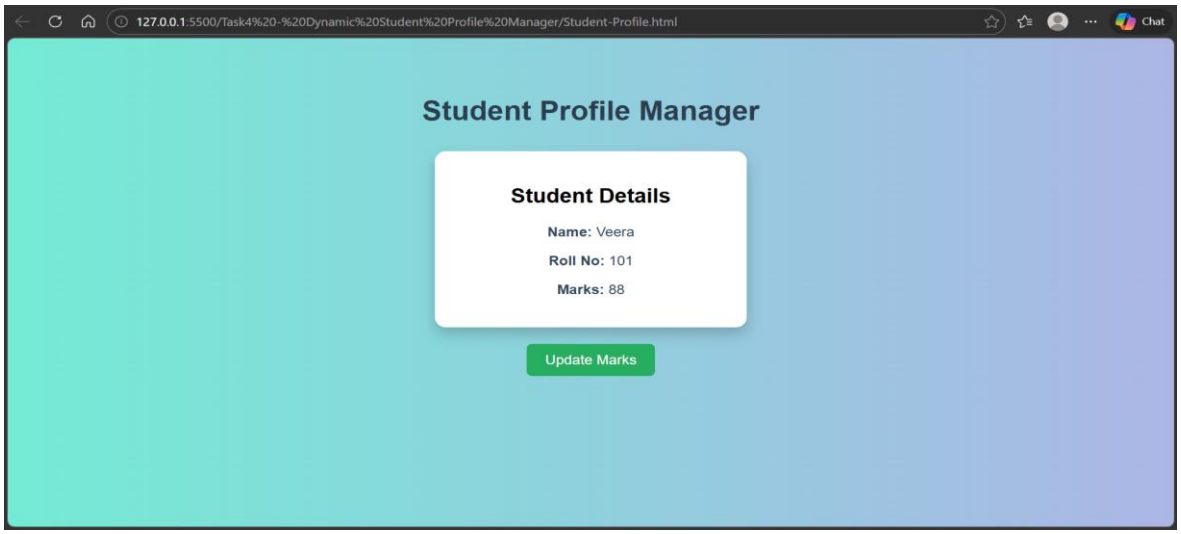
```css
/* Page styling */
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(to right, #74ebd5, #acb6e5);
    text-align: center;
    padding: 40px;
}

/* Heading */
h1 {
    color: #2c3e50;
    margin-bottom: 30px;
}

/* Student profile box */
#profileBox {
    background-color: #ffffff;
    width: 300px;
    margin: auto;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
}

/* Text inside profile */
#profileBox p {
    font-size: 16px;
    color: #34495e;
}
```

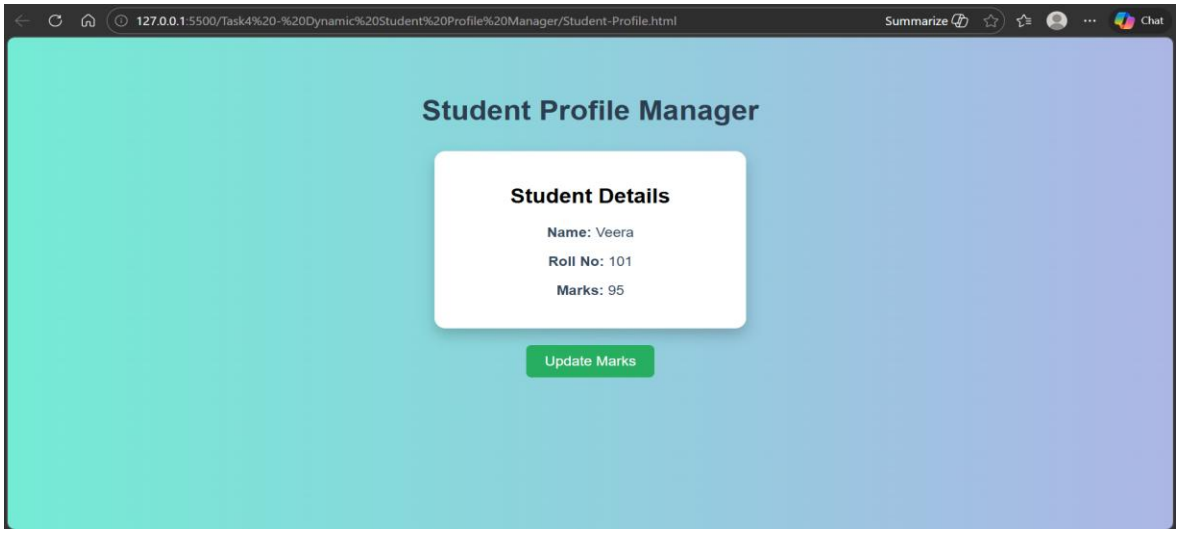
Task4 - Dynamic Student Profile Manager > # style.css > button:hover

```
/* Button styling */
button {
    margin-top: 20px;
    padding: 10px 20px;
    background-color: #2ecc71;
    border: none;
    border-radius: 6px;
    color: white;
    font-size: 16px;
    cursor: pointer;
}

/* Button hover effect */
button:hover {
    background-color: #27ae60;
}
```

Output Screenshot:

The screenshot shows a **Student Profile Manager**. It displays student details including Name (Veera), Roll No (101), and Marks (88). Below the details, there is an **"Update Marks"** button. The button is used to modify or update the student's marks dynamically.





The "Update Marks" button at the bottom likely triggers a JavaScript function to change the displayed value from the initial 80 to the current 95. This is a classic example of **DOM manipulation**, where clicking a button updates a specific part of the webpage's content in real-time.